



North South University

Course name : CSE 225
Section : 12
Semester : Summer 24
HW Topic : Time complexity & Growth rate
Date : 11-09-2024

Submitted To

Dr. Md. Ahsan Habib (AHbb)

Submitted by:

NAME : Mohammad Tushar Abdullah

ID : 2323766042

Answer to the question no : 01

a) The time complexity of this algorithm is $O(n^2)$

b) In this algorithm there is two nested loops

The outer loop runs from 1 to n means $i = 1$ to $i = n$. & the inner loop is also running from 1 to n means $j = 1$ to $j = n$.

Since the outer loop is running n times and the inner loop also runs n time so, the total number of iteration is $n \times n = n^2$

Therefore, the time complexity is : $O(n^2)$

c) Here the worst case scenario will happen if the algorithm needs to check the maximum number of comparisons before finding the same element.

It can be if There is no common elements. Then the algorithm will check every pair.

Let two arrays are :

A = [1, 2, 3, 4, 5]

B = [6, 7, 8, 9, 10]

Algorithm :

for $i = 1$ to n :

 for $j = 1$ to n :

 if $A[i] == B[j]$:

 return True

 return False

It will check by comparing every element of A with every element of B, and none of them will satisfy the condition $A[i] == B[j]$. Hence the algorithm will run $5 \times 5 = 25$ times but will get no matches.

Answer to the question no : 02

a) The outer loop runs from 0 to (n-1). So it iterates n times.

Inner loop runs from (i+1) to (n-1) so the number of iterations of the inner loop decreases as i increases. For each i , the number of iterations of the inner loop is

$$n - (i+1)$$

$$\text{Total iterations} = (n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1) / 2$$

Therefore, the time complexity in the worst case is $O(n^2)$

Because the total number of iterations is proportional to n^2 .

b) In the worst case, the algorithm will perform approximately $n(n-1) / 2$ comparisons before returning 'False'.

$$\text{Number of comparisons} = n(n-1) / 2$$

$$= \{ 1000000 (1000000 - 1) \} / 2$$

$$= 499999500000$$

So, the algorithm would perform around 499.999 billion comparisons in the worst case when $n = 1000000$.

Answer to the question no : 03

Ranking the following complexities from fastest growing to slowest:

1. $O(2^n)$: This is exponential growth, which grows the fastest and becomes impractical for large inputs.
2. $O(n^2)$: This is quadratic growth. It grows slower than exponential but faster than other common complexities.
3. $O(n \log n)$: This is often seen in algorithms like merge sort and is more efficient than quadratic but slower than logarithmic.
4. $O(\log n)$: This is the logarithmic growth, which is very efficient and grows the slowest, making it preferable for large input sizes.

Q : Which complexity would be preferable for large input sizes and why?

Ans : For large input sizes, $O(\log n)$ is the best choice. This is because it grows very slowly as the input size gets bigger. Algorithms with logarithmic time complexity, like binary search, reduce the problem size a lot with each step, so they work very fast even with huge amounts of data. This makes them perfect for handling large datasets or big problems efficiently.